

Supervised learning (II): Classification (I)

- generative models**
- k-nearest neighbors**

BIOST 2155

Class 5

September 25th, 2026

The confusion matrix

Quantifying performance for classification problems

Overall Accuracy: proportion of total predictions that were correct. <i>“Overall, how often is the model right?”</i>	$(\text{TP} + \text{TN}) / (\text{TP} + \text{TN} + \text{FP} + \text{FN})$	Classify as $\hat{Y} = 1$ if $\hat{P}(Y = 1 \mid X = x) > 0.5$		<i>Predicted Label</i>	
Sensitivity: proportion of actual positives that were correctly identified. <i>"Of all the true positive cases, how many did we catch?"</i>	$\text{TP} / (\text{TP} + \text{FN})$			<i>Positive</i>	<i>Negative</i>
Specificity: proportion of actual negatives that were correctly identified. <i>"Of all the actual negative cases, how many did we correctly rule out?"</i>	$\text{TN} / (\text{TN} + \text{FP})$	Actual label	Positive	True Positive	False Negative
Youden's J Index: summarizes the diagnostic ability, balancing sensitivity and specificity	Sensitivity + Specificity - 1 <i>(Express using TP, TN, FP, FN)</i>		Negative	False Positive	True Negative

The confusion matrix (contd.)

Quantifying performance for classification problems

What if we lower the threshold to 0.4?

- Sensitivity would likely increase.
- Specificity would likely decrease.

Sensitivity: proportion of actual positives that were correctly identified. <i>"Of all the true positive cases, how many did we catch?"</i>	TP / (TP + FN)
Specificity: proportion of actual negatives that were correctly identified. <i>"Of all the actual negative cases, how many did we correctly rule out?"</i>	TN / (TN + FP)

		Predicted Label	
		Positive ⬆	Negative ⬇
Actual label	Positive	True Positive	False Negative
	Negative	False Positive	True Negative

The confusion matrix (contd.)

Quantifying performance for classification problems

What if we lower the threshold to 0.4?

- Sensitivity would likely increase.
- Specificity would likely decrease.

How do we evaluate a model's performance across all possible thresholds?

Receiver Operating Characteristic (ROC) Curve

		Predicted Label	
		Positive 	Negative 
Actual label	Positive	True Positive	False Negative
	Negative	False Positive	True Negative

The confusion matrix (contd.)

Quantifying performance for classification problems

ROC curve visualizes the performance of a classifier by plotting the **True Positive Rate** (Sensitivity) against **False Positive Rate** (1 - Specificity) at every possible threshold.

Y-axis: **Sensitivity** vs X-axis: **1 - Specificity**

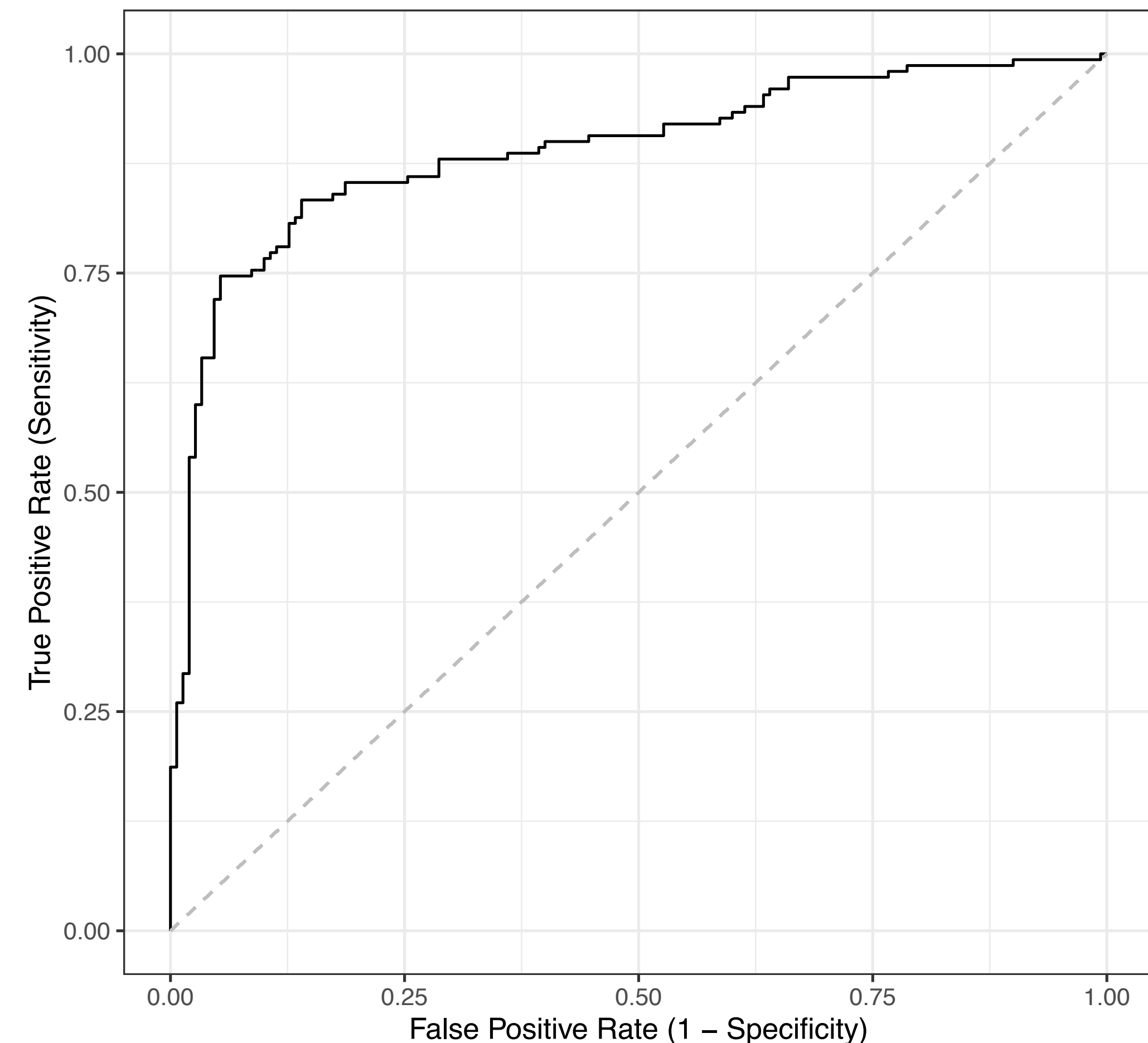
A better model has a curve that is closer to the top-left corner (high sensitivity, low false positive rate).

AUC: Area Under the (ROC) Curve. *Measures the entire two-dimensional area underneath the entire ROC curve. It provides an aggregate measure of performance across all classification thresholds.*

- **AUC = 1:** A perfect classifier.
- **AUC = 0.5:** A model with no discrimination ability, equivalent to random guessing (the diagonal line).
- **$0.5 < \text{AUC} < 1$:** The model has some ability to discriminate. The closer the AUC is to 1, the better the classifier.

ROC Curve for Logistic Regression

Area Under Curve (AUC): 0.891



Generative models for classification

Why not just stick with logistic (multinomial) regression?

- In logistic (or multinomial) regression approach, we model $P(Y = k \mid X)$ for $k \in \mathcal{Y}$, $\mathcal{Y}$ is the set of values that Y can take.
- In the generative model approach, we
 - **Step 1:** Model $P(X \mid Y = k)$ separately for each value of $k \in \mathcal{Y}$
 - **Step 2:** Use Bayes theorem to ‘flip’ step 1 into estimates for $P(Y \mid X)$
- **For substantial separation between the classes, $\hat{\beta}_{LOGIS}$ are unstable.**
- If X is approximately normal for each $k \in \mathcal{Y}$, then this approach is better for small n

Generative models for classification

Recap of Bayes theorem

- In the generative model approach, we
 - **Step 1:** Model $P(X \mid Y = k)$ separately for each value of $k \in \mathcal{Y}$
 - **Step 2:** Use Bayes' theorem to 'flip' step 1 into estimates for $P(Y = k \mid X)$

- Bayes' theorem

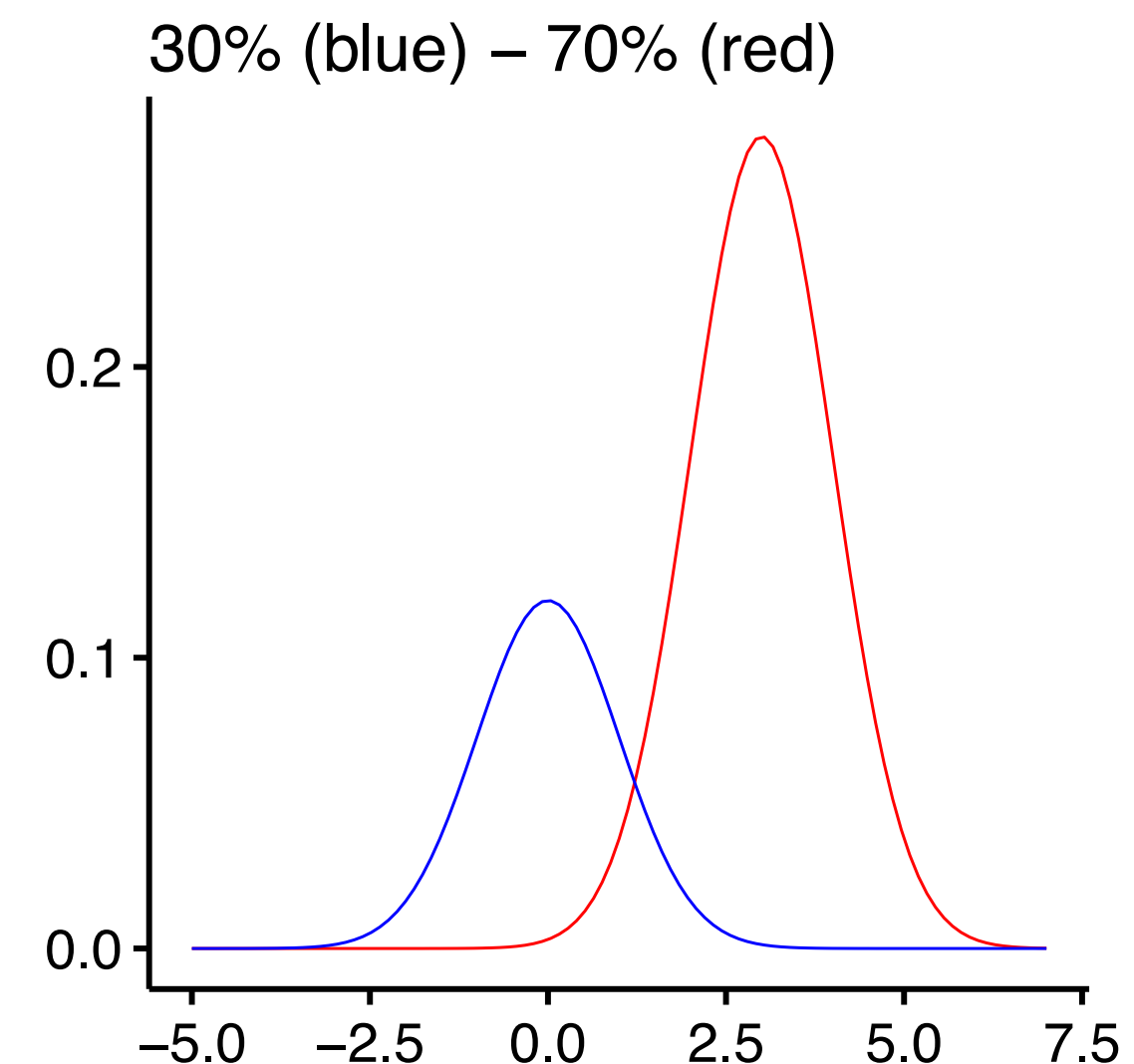
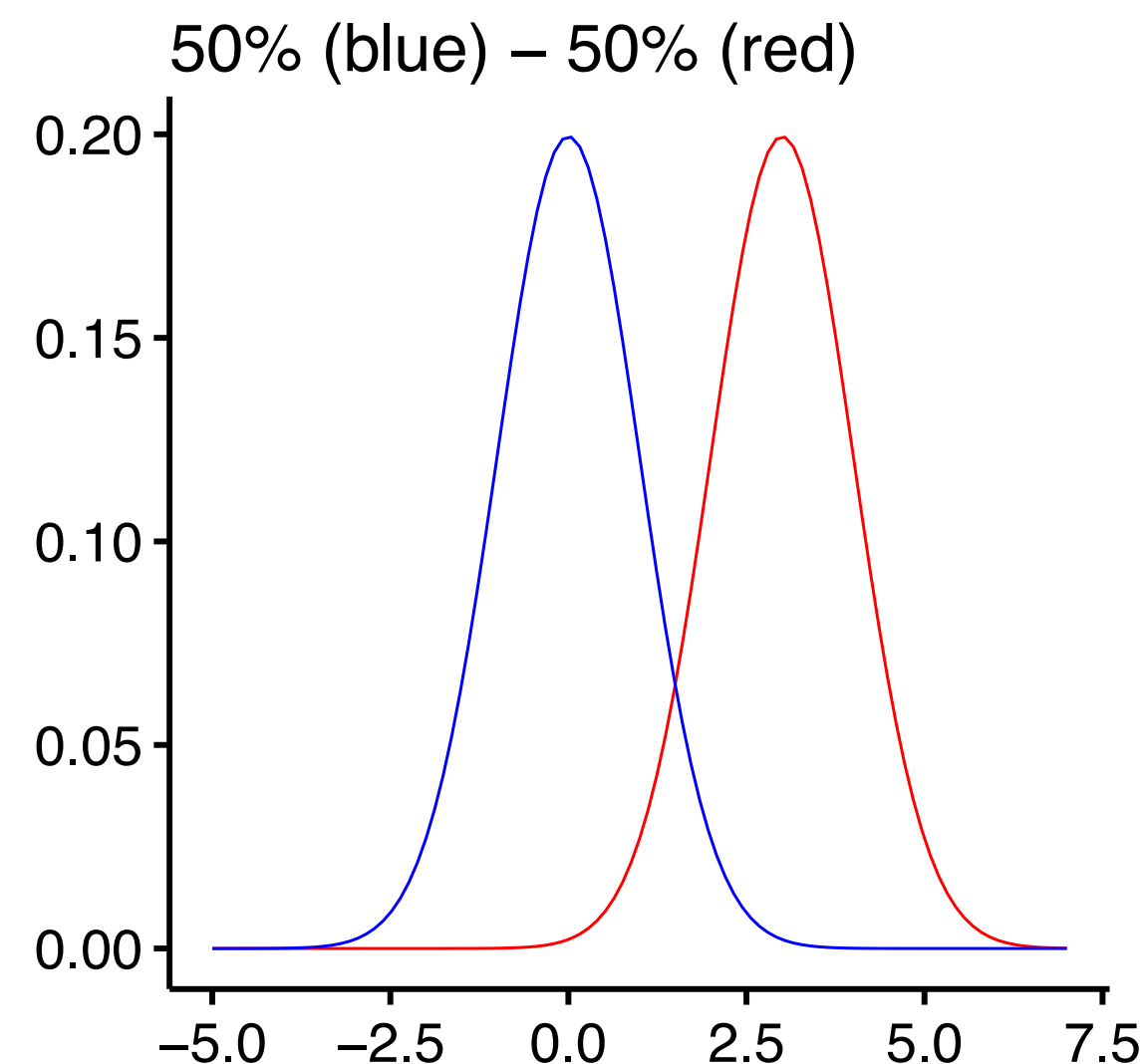
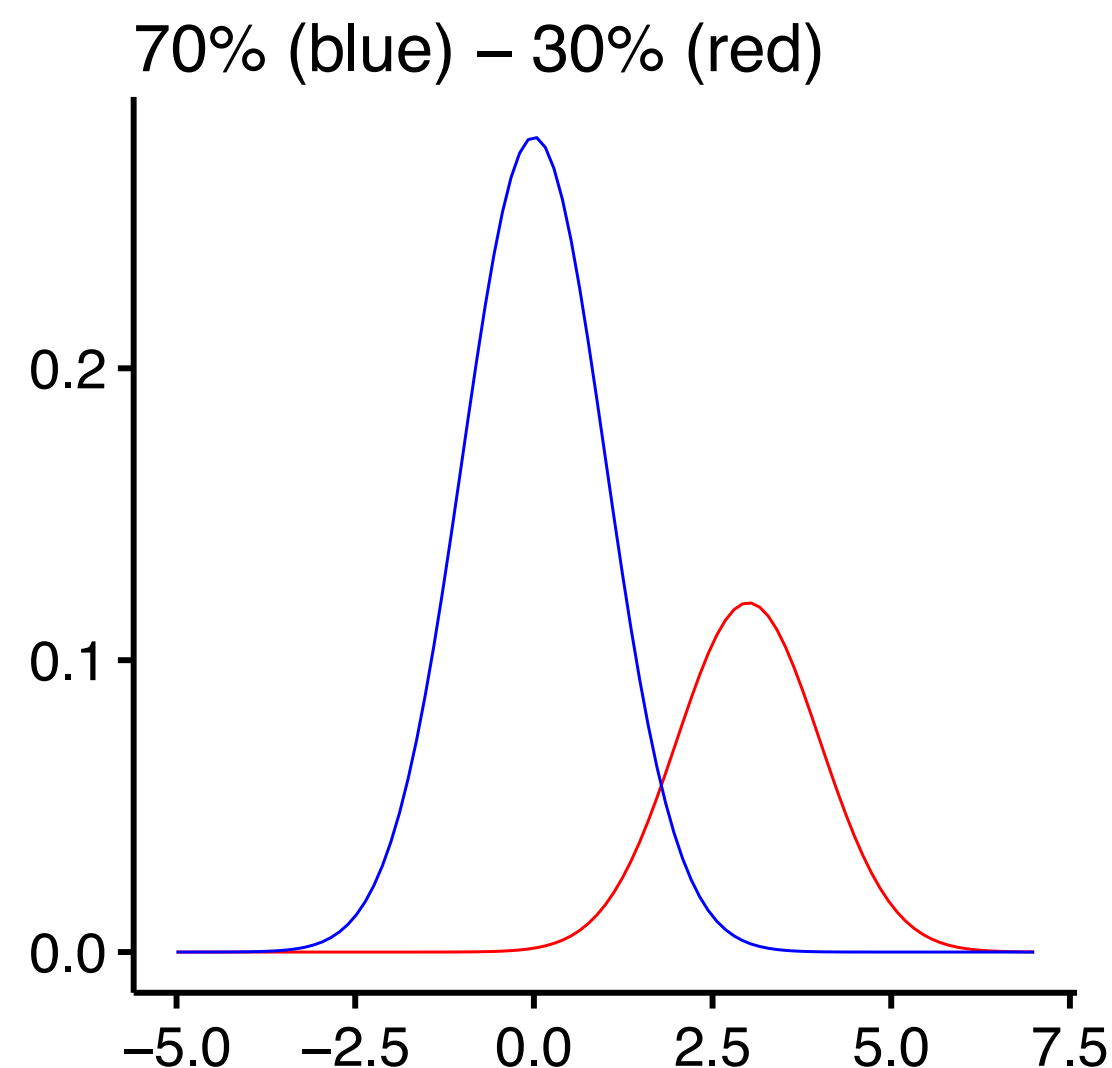
$$\mathbb{P}(Y = k \mid X = \mathbf{x}) = \frac{\mathbb{P}(X = \mathbf{x} \mid Y = k) \cdot \mathbb{P}(Y = k)}{\mathbb{P}(X = \mathbf{x})} = \frac{\mathbb{P}(X = X \mid Y = k) \cdot \mathbb{P}(Y = k)}{\sum_{k \in \mathcal{Y}} \mathbb{P}(X = X \mid Y = k) \cdot \mathbb{P}(Y = k)}$$

- $\mathbb{P}(X = X \mid Y = k) = f_k(\mathbf{x})$ describes density of predictors in class (label) k
- $\mathbb{P}(Y = k) = \pi_k$ is the marginal probability of class (label) k

Generative models for classification

Recap of Bayes theorem (contd.)

- **Step 1:** Model $P(X \mid Y = k)$ separately for each value of $k \in \mathcal{Y}$
- **Step 2:** Use Bayes' theorem to 'flip' step 1 into estimates for $P(Y = k \mid X)$
- **Step 3:** Predict label $\hat{k}(x) = \operatorname{argmax}_k \mathbb{P}(Y = k \mid X = x)$



Classify to the highest density!!

Linear discriminant analysis

Starting step: $p = 1$ and using Gaussian distributions for $f_k(x)$

- Starting step: $f_k(x) = \sigma^{-1} \phi((x - \mu_k)/\sigma)$ (**can be other distributions as well**)
 - $X \mid Y = k \sim \text{normal}$; **different means (known)**, **same variance (known)**.
 - So, $p_k(x) \propto \pi_k \exp(- (x - \mu_k)^2 / 2\sigma^2)$
 - **Decision rule:** Assign to class k_0 if $p_{k_0}(x) > p_k(x)$ for all $k \neq k_0$
 - $p_{k_0}(x) > p_k(x) \iff \delta_{k_0}(x) > \delta_k(x)$
 - **Discriminant function** $\delta_k(x) = x \frac{\mu_k}{\sigma^2} - \frac{\mu_k^2}{2\sigma^2} + \log(\pi_k)$

For real data: we use plug-in estimates

$$\hat{\pi}_k = n_k/n \quad \hat{\mu}_k = \frac{1}{n_k} \sum_{i:y_i=k} x_i \quad \hat{\sigma}^2 = \sum_{k=1}^K \frac{n_k - 1}{n - K} \hat{\sigma}_k^2$$

Linear discriminant analysis

Next: General p and using Gaussian distributions for $f_k(\mathbf{x})$

$$f_k(\mathbf{x}) = \frac{1}{(2\pi)^{p/2} |\mathbf{\Sigma}|^{1/2}} e^{-\frac{1}{2}(\mathbf{x}-\boldsymbol{\mu}_k)^T \mathbf{\Sigma}^{-1}(\mathbf{x}-\boldsymbol{\mu}_k)}$$

$X \mid Y = k \sim$ **multivariate** normal. **Different means and same variance.**

But for a set of covariates, not just one

$$\delta_k(\mathbf{x}) = \mathbf{x}^T \mathbf{\Sigma}^{-1} \boldsymbol{\mu}_k - \frac{1}{2} \boldsymbol{\mu}_k^T \mathbf{\Sigma}^{-1} \boldsymbol{\mu}_k + \log \pi_k$$

For real data, we use plug-in estimates (again)

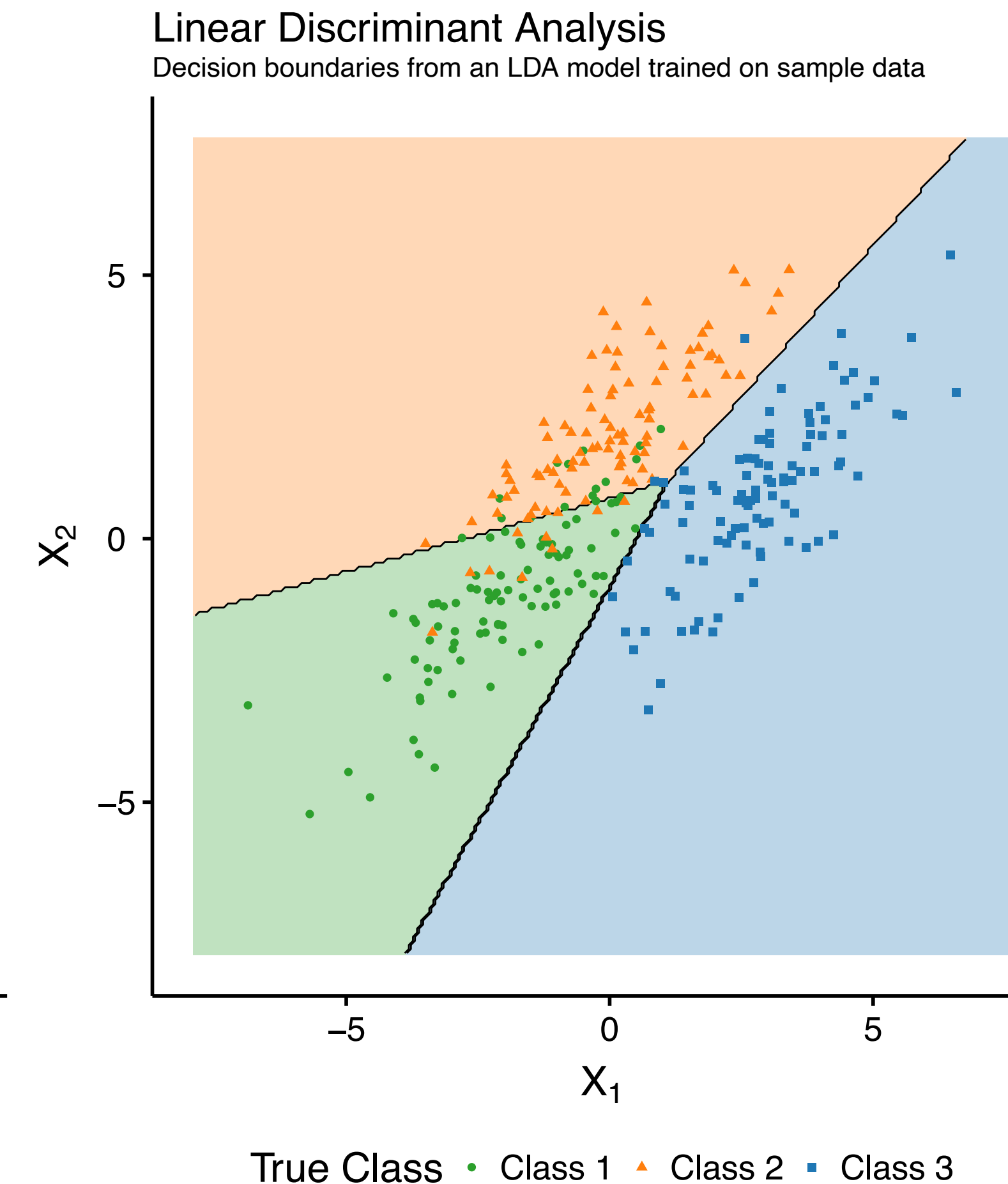
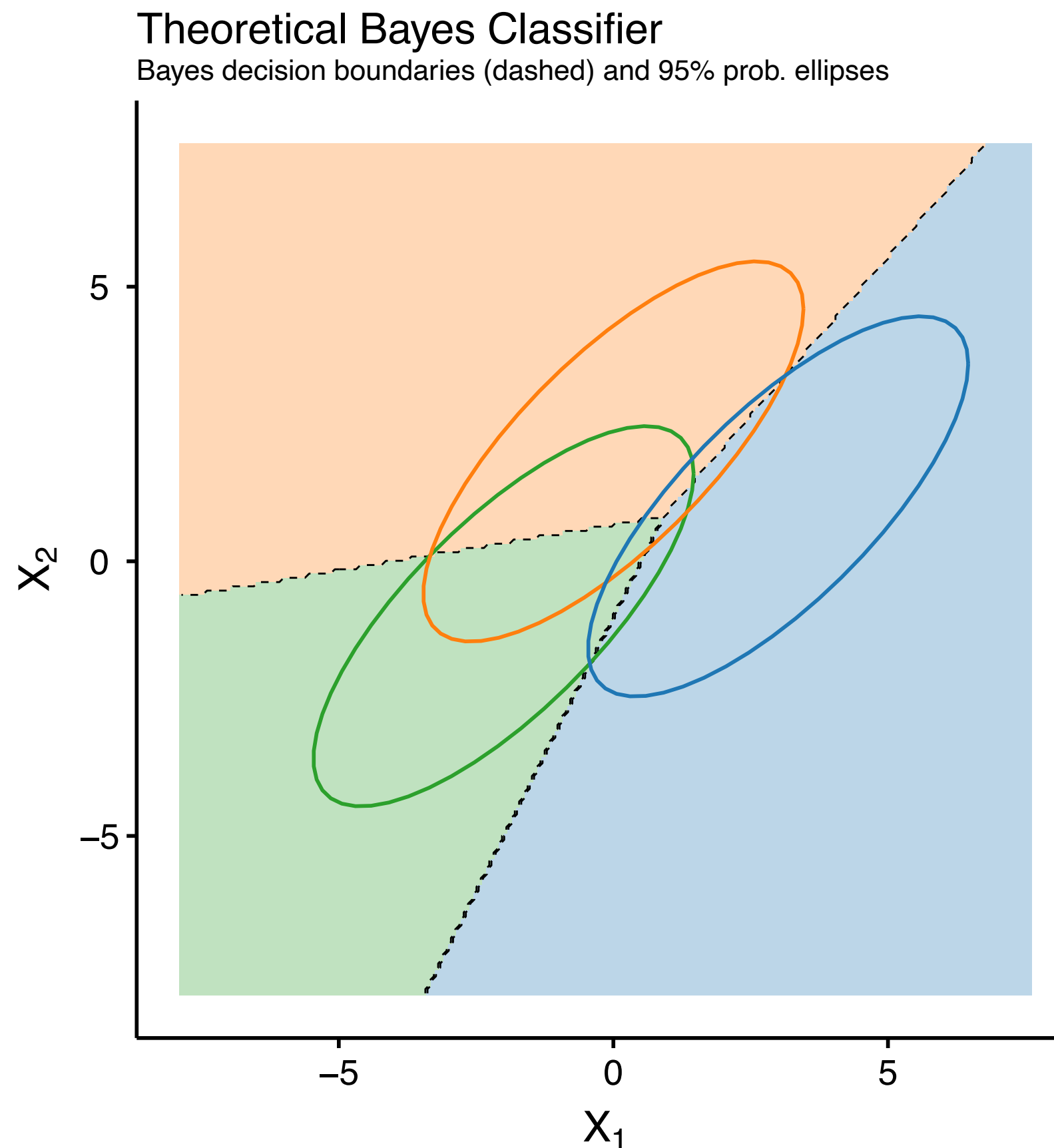
Linear discriminant analysis

Illustration: General p and using Gaussian distributions for $f_k(\mathbf{x})$

Three distinct classes (green, orange, blue), each with a bivariate Gaussian distribution. Each class has its own mean (center) but share a common covariance matrix

Left panel: theoretical "best case" classifier, known as the Bayes decision boundary (dashed lines). This ideal boundary is calculated using the true, known mean and covariance of each class.

Right panel: the true parameters are unknown. LDA model is trained on data. The resulting linear lines are the decision boundaries estimated by the LDA model.



Quadratic discriminant analysis

General p , using Gaussian distributions for $f_k(\mathbf{x})$

$$f_k(\mathbf{x}) = \frac{1}{(2\pi)^{p/2} |\boldsymbol{\Sigma}_k|^{1/2}} e^{-\frac{1}{2}(\mathbf{x} - \boldsymbol{\mu}_k)^T \boldsymbol{\Sigma}_k^{-1} (\mathbf{x} - \boldsymbol{\mu}_k)}$$

$X \mid Y = k \sim$ **multivariate** normal. **Different means and different variances.**

$$\delta_k(\mathbf{x}) = -\frac{1}{2}(\mathbf{x} - \boldsymbol{\mu}_k)^T \boldsymbol{\Sigma}_k^{-1} (\mathbf{x} - \boldsymbol{\mu}_k) - \frac{1}{2} \log |\boldsymbol{\Sigma}_k| + \log \pi_k$$

For real data, we use plug-in estimates (again)

Quadratic discriminant analysis

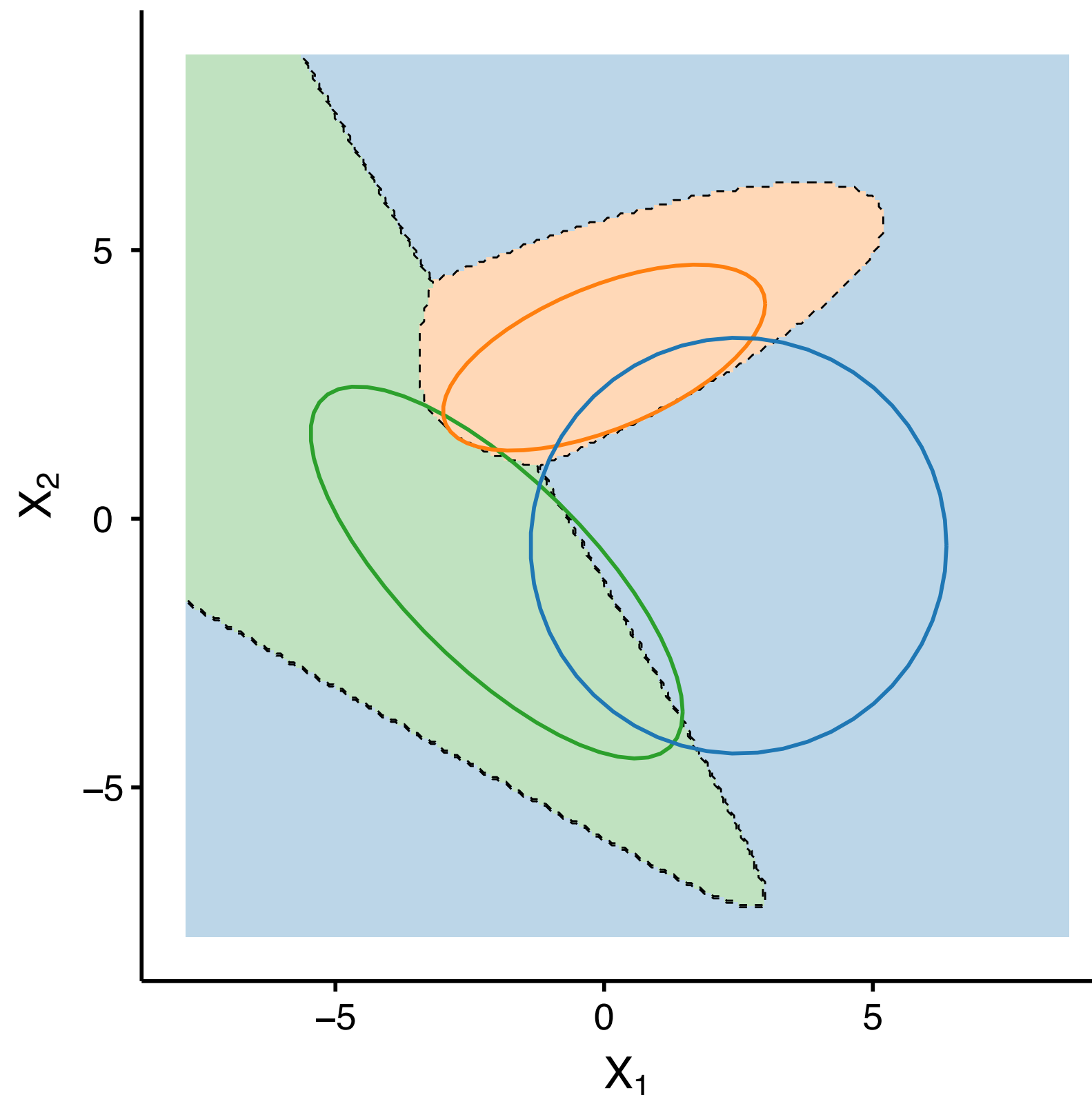
Illustration: General p , using Gaussian distributions for $f_k(x)$

Three distinct classes (green, orange, blue), each with a bivariate Gaussian distribution. Each class has its own unique mean (center) and covariance matrix

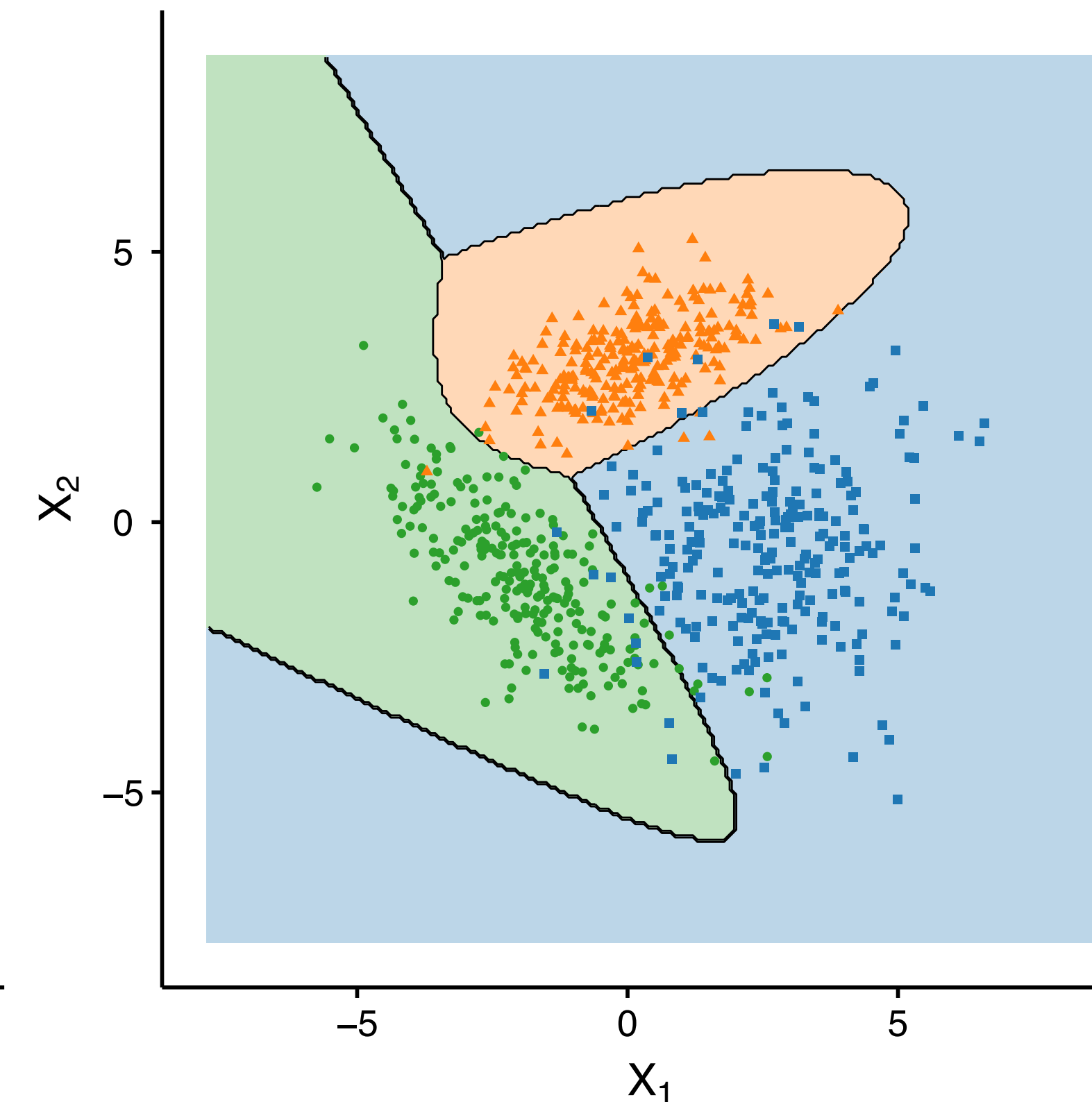
Left panel: theoretical "best case" classifier, known as the Bayes decision boundary (dashed lines). This ideal boundary is calculated using the true, known mean and covariance of each class.

Right panel: the true parameters are unknown. QDA model is trained on data. The resulting parabolic curves are the decision boundaries estimated by the QDA model.

Theoretical Bayes Classifier (QDA)
Bayes decision boundaries are now quadratic



Quadratic Discriminant Analysis
Decision boundaries from a QDA model trained on sample data



True Class • Class 1 ▲ Class 2 ■ Class 3

Naive Bayes - one more generative model

A different **distributional assumption**

Same three steps as LDA and QDA — only $f_k(\mathbf{x})$ changes:

1. Model $f_k(\mathbf{x}) = \mathbb{P}(\mathbf{x} \mid Y = k)$ in each class k .
2. Flip it with Bayes' theorem to get $\mathbb{P}(Y = k \mid \mathbf{x})$.
3. Predict $\hat{k}(\mathbf{x}) = \arg \max_k \pi_k f_k(\mathbf{x})$.

Assume the p predictors are independent within each class: $f_k(\mathbf{x}) = \prod_{j=1}^p f_{kj}(x_j)$.

No covariance matrix to estimate!

QDA needs a full matrix Σ_k **per class** — that is where it runs out of data.

Naive Bayes drops the off-diagonals, keeping only p variances per class

Where it sits: LDA → QDA → naive Bayes

All three fit into the same generative framework, differing only in the **covariance assumption**

Method	Covariance	Boundary	Params/class
LDA	Shared	linear	??
QDA	Class-specific	quadratic	??
Naive Bayes	Independence (assumed)	axis-aligned	??

Assume the p predictors are independent within each class: $f_k(\mathbf{x}) = \prod_{j=1}^p f_{kj}(x_j)$.

each f_{kj} is a **1-D** density — estimate it any way

(Gaussian, kernel, or class proportions for a categorical predictor).

Naive Bayes takes **mixed-type predictors** natively; LDA and QDA cannot.

k-Nearest Neighbors

A non-parametric classification (*and regression*) tool

- For a new observation (x_0, y) , it finds the K -most similar training observations (i.e., neighbors) and makes a prediction based on them.
- **How it works:**
 - **Step 1:** fix K .
 - **Step 2:** find neighbor set $\mathcal{N}_0$. *Identify the K training data points that are closest to the new, unseen observation x_0*
 - **Step 3:** majority vote. Assign the new observation to the class that is most common among its K neighbors.

Non-parametric: It makes no assumptions about the functional form of the boundary. Can find highly non-linear boundaries.

Instance-based: No model is explicitly trained; it simply uses the entire training dataset at prediction time.

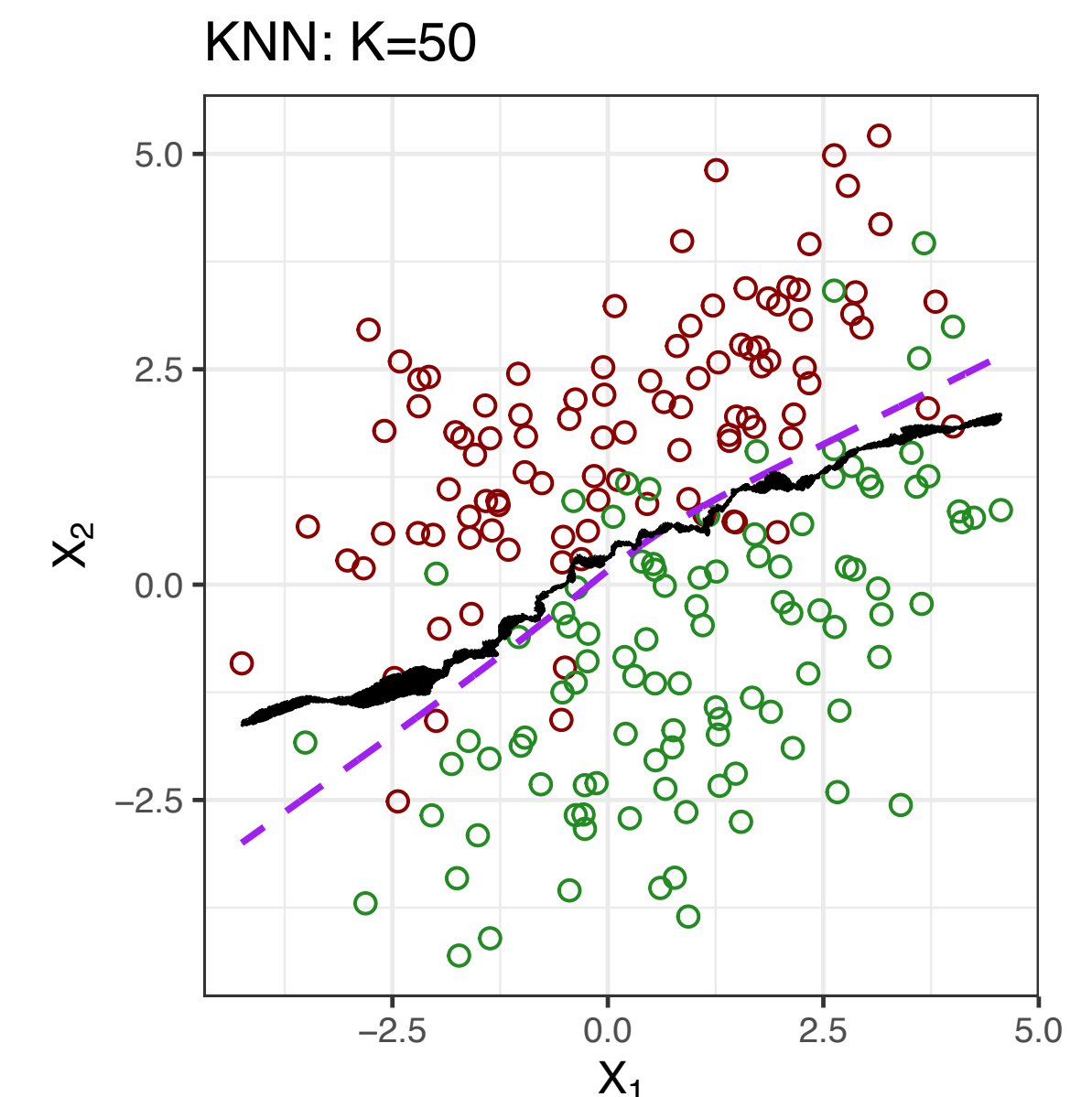
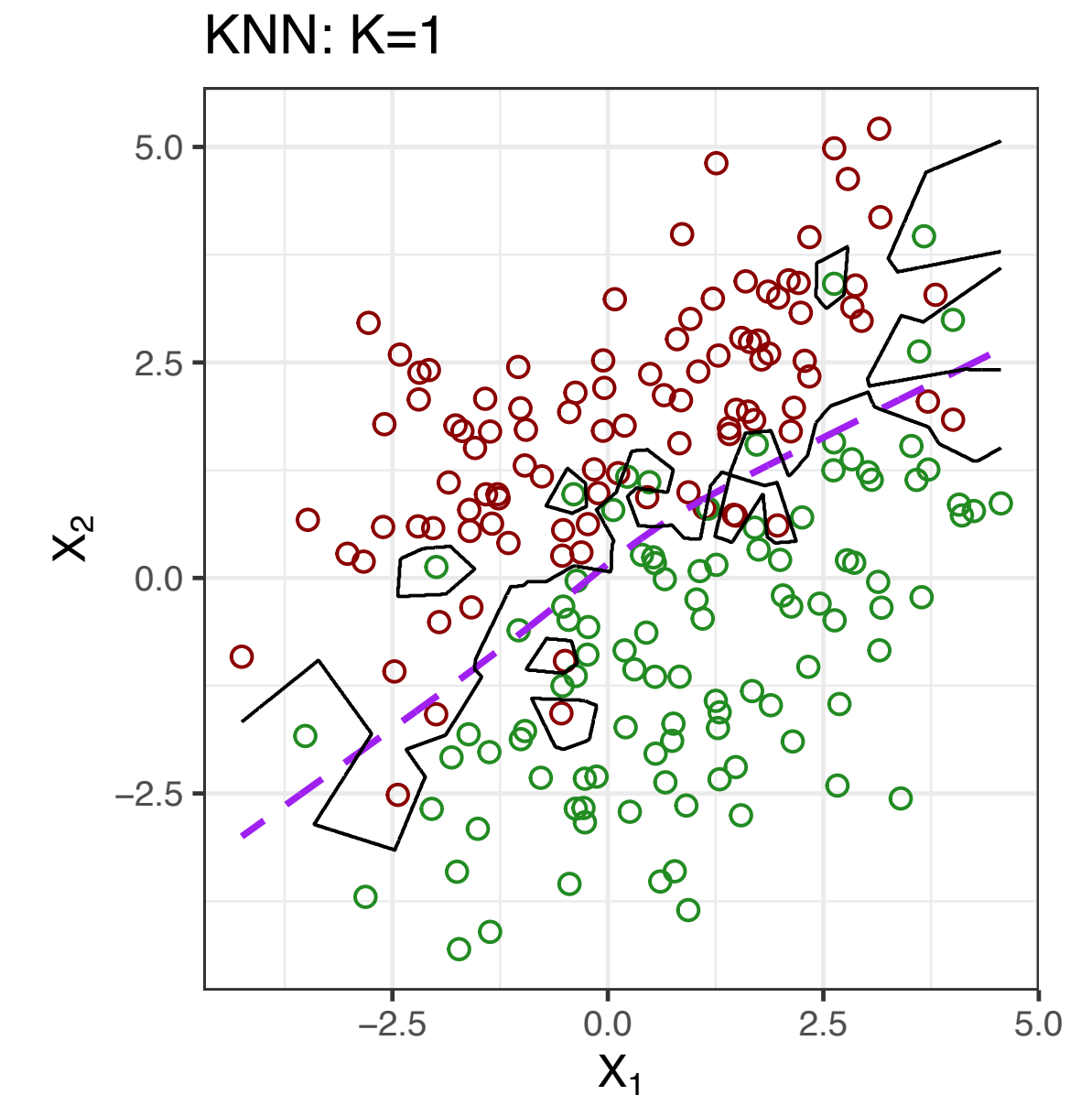
$$\mathbb{P}(Y = j | X = x_0) = \frac{1}{K} \sum_{i \in \mathcal{N}_0} I(y_i = j)$$

$$\hat{Y} = \operatorname{argmax}_{j \in \mathcal{Y}} \mathbb{P}(Y = j | X = x_0)$$

k-Nearest Neighbors (contd.)

Choosing K and the Bias-Variance Trade-Off

- Low K (e.g., $K=1$):
 - **High Flexibility:** The decision boundary is very jagged and follows the training data closely.
 - **Low Bias, High Variance:** The model is prone to overfitting by capturing noise in the data.
- High K (e.g., $K=100$):
 - **Low Flexibility:** The decision boundary becomes much smoother and resembles a linear boundary.
 - **High Bias, Low Variance:** The model might be too simple and underfit the data.
- **How to Choose the Optimal K: cross-validation!!**



Post-class debrief

Today we discussed...

This lecture covered several supervised learning techniques for classification, methods for evaluating their performance, and the trade-offs involved.

- **Confusion Matrix** is used to quantify a classifier's performance using metrics like Sensitivity, Specificity, and Overall Accuracy
- **Receiver Operating Characteristic (ROC)** curve visualizes the trade-off between the True Positive Rate (Sensitivity) and the False Positive Rate (1 - Specificity) across all possible thresholds
- **Area Under the Curve (AUC)** provides a single aggregate score for model performance, where an AUC of 1 is a perfect classifier and 0.5 is equivalent to random guessing

- **Generative Models:** estimate the probability distribution of the predictors for each class and use Bayes' theorem.
 - **Linear Discriminant Analysis (LDA):** Assumes predictors follow a Gaussian distribution with a different mean for each class but a common covariance matrix. This results in linear decision boundaries
 - **Quadratic Discriminant Analysis (QDA):** More flexible than LDA, it assumes predictors follow a Gaussian distribution where each class has its own unique mean and covariance matrix. This results in quadratic (curved) decision boundaries
- **k-Nearest Neighbors (k-NN):** This is a non-parametric, instance-based algorithm that classifies a new data point based on a "majority vote" of its K closest neighbors in the training data
 - **Choosing K:** The choice of K involves a bias-variance trade-off. Cross-validation is used to find the optimal K



That's all Folks!